

DOCUMENTACIÓN DEL MÓDULO 4 – SYNTHCITY (SPRINT 2)

1. OBJETIVO DEL SPRINT

Construir la primera interfaz gráfica operativa del sistema e introducir la persistencia real mediante JDBC. El sistema debe pasar de ser una calculadora de resultados por texto a una aplicación que muestre la ciudad visualmente sobre su rejilla (PanelCiudad), muestre su estado analítico básico (PanelResumenSistema) y permita guardar los resultados finales en una base de datos (JDBC).

Además, la persistencia debe registrar los resultados visibles y también un conjunto mínimo de variables analíticas (densidad, porcentaje de actividad, ratios energético y de cobertura de servicios, contaminación, estabilidad básica y score de viabilidad) para que en sprints posteriores se puedan usar en predicciones basadas en datos.

2. PLANIFICACIÓN Y ORGANIZACIÓN DEL EQUIPO

El equipo de 4 personas se organizó según las responsabilidades definidas en el documento:

- **Persona 1** – Responsable de la ventana principal y del panel visual del grid. Clases: VentanaPrincipal, PanelCiudad. Álvaro.
- **Persona 2** – Responsable del panel resumen del sistema (estado analítico). Clase: PanelResumenSistema. Roberto.
- **Persona 3** – Responsable de la persistencia básica con JDBC. Clases: DatabaseManager, ResultadoRepository. Víctor.
- **Persona 4** – Responsable del controlador GUI, del resumen textual auxiliar y de las pruebas de integración. Clases: ControladorGUI, PresentadorCiudad, SalidaTexto. Alexandre.

Coordinación:

- Daily Scrum de 5 minutos.

- Integración continua: se acordó que la integración no se dejaría para el final; oír lo que desde el Jueves estuvimos trabajando en integrar para que se pudesen realizar las pruebas unitarias.
- Ramas Git por Persona Integramos todo en el PC de Alexandre antes de subirlo a una única rama.

3. DECISIONES DE DISEÑO OBLIGATORIAS (SEGÚN ESPECIFICACIÓN)

- **JavaFX para la GUI:** Mejor integración con grid, paneles y eventos que Swing; separación limpia mediante BorderPane, GridPane, VBox, HBox.
- **BorderPane como contenedor principal:** Organización clara: centro grid, derecha resumen, abajo botones.
- **GridPane para el dibujo de la ciudad:** Representación natural de una rejilla bidimensional; cada celda es un StackPane con fondo y texto, se ha añadido también divisores más claros y sombreado por cuestión de estilo, no era obligatorio para este sprint.
- **Separación estructural/analítica:** PanelCiudad solo dibuja estructura; PanelResumenSistema solo muestra análisis (evaluación y predicción). Ninguno recalcula nada.
- **Persistencia mínima obligatoria:** Se guardan: nombre ciudad, nivel evaluación, score, mensaje evaluación, tendencia predicha, score predicho, mensaje predicción, fecha. Además se almacenan variables analíticas: densidad, porcentaje actividad, ratio energético, ratio cobertura servicios, contaminación, estabilidad básica, score viabilidad.
- **Recuperación básica** – Se implementa al menos obtenerUltimoResultado() o listado simple de registros.

4. ARQUITECTURA Y CLASES OBLIGATORIAS

Capa visual

- **VentanaPrincipal:** Contenedor BorderPane con PanelCiudad, PanelResumenSistema y botones. No contiene lógica.
- **PanelCiudad:** Extiende GridPane. Dibuja la ciudad recibida. Cada celda es un StackPane con Rectangle de fondo y texto. Distingue tipos de bloque y estado activo/inactivo.
- **PanelResumenSistema:** Extiende VBox. Muestra nombre ciudad, tipo estructural, ocupación, densidad, nivel evaluación, score, mensaje evaluación, tendencia predicha, score predicho, mensaje predicción.

Capa de coordinación

- ControladorGUI: Coordina la GUI, recibe Ciudad, ResultadoEvaluacion, PredictionResult; actualiza los paneles; gestiona acciones de botones (refrescar, guardar, limpiar); invoca la persistencia.

Capa de persistencia

- DatabaseManager – Centraliza URL, usuario, contraseña y método getConnection(). Incluye inicializarTablaResultados() para crear/verificar la tabla.
- ResultadoRepository – se encarga de guardar el resultado (inserta registro) y recuperar resultados básicos (listarResultadosBasicos(), obtenerUltimoResultado()).

Clases auxiliares

- ResumenTextoSistema – Construye un resumen textual legible del sistema para depuración o salida de apoyo.
- SalidaTexto – Encapsula el resumen textual.

5. INVARIANTES OBLIGATORIAS DEL MÓDULO

Invariantes visuales

- El grid debe corresponder exactamente a la ciudad recibida.
- El panel resumen debe corresponder exactamente a la evaluación y predicción recibidas.
- Lo guardado debe coincidir con lo mostrado.
- No deben faltar campos esenciales en la base de datos.

Invariantes de arquitectura

- PanelCiudad representa solo la estructura.
- PanelResumenSistema representa solo el análisis.
- ControladorGUI coordina, pero no contiene lógica de simulación o evaluación.
- ResultadoRepository persiste, pero no formatea ni interpreta.

6. API PÚBLICA PRINCIPAL (RESUMEN)

VentanaPrincipal

- public VentanaPrincipal()
- public void setControlador(ControladorGUI controlador)

- public void setPanelResumen(PanelResumenSistema panelResumen)
- public void actualizarCiudad(Ciudad ciudad)
- public void mostrar()

PanelCiudad

- public void mostrarCiudad(Ciudad ciudad)
- public void limpiar()
- public void refrescar()

ControladorGUI

- public void mostrarSistema(Ciudad ciudad, ResultadoEvaluacion eval, PredictionResult pred)
- public void refrescarVista()
- public void limpiarVista()
- public void guardarResultadoActual()
- public void cargarUltimoResultado() (opcional)

ResultadoRepository

- public void guardarResultado(Ciudad ciudad, ResultadoEvaluacion eval, PredictionResult pred)
- public List<ResultadoRegistro> listarResultadosBasicos()
- public ResultadoRegistro obtenerUltimoResultado()

resultado de hacer los metodos publicos para que se pueda acceder a ellos dese cualquier otra clase.

7. PLAN DE PRUEBAS Y RESULTADOS

Se ejecutaron pruebas en cuatro categorías, tal como especifica el documento del sprint.

A. Pruebas de GUI

- La ventana se abre correctamente.
- El grid se dibuja con las dimensiones correctas.
- Celdas vacías y ocupadas se distinguen bien.
- Los tipos de bloque se representan con colores y texto.
- Bloques inactivos se ven con opacidad 0.6.

B. Pruebas de estado del sistema

- Nombre de ciudad correcto en resumen.
- Densidad mostrada correcta .
- Nivel de evaluación mostrado correcto.
- Score mostrado correcto.
- Tendencia predicha mostrada correcta .

C. Pruebas de JDBC

- Conexión a base de datos funciona.
- La tabla se crea/verifica correctamente.
- El resultado actual se guarda correctamente.
- El registro guardado contiene los campos correctos (incluidas variables analíticas).
- Se puede recuperar el último resultado (o listado básico).

D. Pruebas de integración

- La GUI no recalcula nada.
- La ciudad mostrada y el resumen pertenecen al mismo estado.
- La aplicación no se rompe al integrar el trabajo de las cuatro personas.

8. CRITERIOS DE ACEPTACIÓN DEL SPRINT

- VentanaPrincipal, PanelCiudad, PanelResumenSistema, ControladorGUI, DatabaseManager, ResultadoRepository implementados.
- La ventana se abre y muestra el grid de la ciudad.
- El panel resumen muestra todos los campos obligatorios (evaluación y predicción).
- Botones de refrescar, guardar y limpiar funcionan (al menos delegan en el controlador).
- Se puede guardar el resultado actual en base de datos (JDBC).
- Se puede recuperar al menos el último resultado guardado (o listado básico).
- Las variables analíticas (densidad, actividad, ratios, etc.) se almacenan en la base de datos.
- No hay lógica de negocio (simulación, evaluación, predicción) dentro de las clases visuales o de persistencia.
- Existe una salida textual auxiliar (para depuración).
- El código está documentado con JavaDoc y se ha probado la integración entre los cuatro miembros.

9. RIESGOS IDENTIFICADOS Y MITIGACIONES

- **Integración tardía de los paneles visuales:** Se acordó integrar primero VentanaPrincipal y PanelCiudad (Persona 1) con un PanelResumenSistema dummy.
- **JDBC no funcione en todos los entornos:** Se usó una base de datos embebida (H2) para pruebas, con opción a MySQL en producción.
- **El controlador se llene de lógica:** Se revisó en las daily que ControladorGUI solo coordinara, no recalculara.
- **Falta de tiempo para la recuperación de datos:** Se implementó al menos obtenerUltimoResultado() como mínimo exigido.

10. LECCIONES APRENDIDAS (RETROSPECTIVA)

Puntos fuertes

- La separación estricta entre PanelCiudad (estructura) y PanelResumenSistema (análisis) evitó que la GUI invadiera la lógica del dominio.
- Usar BorderPane con regiones claras facilitó la integración entre Persona 1 y Persona 2.
- La decisión de guardar variables adicionales (densidad, ratios, etc.) prepara el sistema para el Sprint 3.

Áreas de mejora

- El placeholder del panel resumen al inicio resultó confuso la próxima vez se definirá un dummy con el mismo ancho.
- La conexión JDBC causó problemas de rutas.
- El método setPanelResumen se corrigió para que acceda correctamente al VBox interno.

11. CONCLUSIONES

El Módulo 4 del Sprint 2 cumple todos los objetivos y está listo para ser implementado